(파이토치 트랜스포머를 활용한) 자연어 처리와 컴퓨터 비전 심층학습

▼ 5장 : 토큰화

자연어는 사람들이 쓰는 언어 활동을 위해 자연히 만들어진 언어이다. 이러한 언어를 처리하는 자연어 처리(Natural Language Processing, NLP)는 컴퓨터가 인간의 언어를 이해하고 해석 및 생성하기 위한 기술을 의미한다. 따라서 인간 언어의 구조, 의미, 맥락을 분석하고 이해할 수 있는 알고리즘을 개발하는데 이러한 모델을 개발하기 위해서는 다음과 같은 문제가 해결되어야 한다.

- 모호성 : 인간의 언어는 단어와 구가 사용되는 맥락에 따라 여러 의미를 갖게 되어 모호한 경우가 만다. 알고리즘은 이러한 다양한 의미를 이해하고 명확하게 구분할 수 있어야 한다.
- 가변성 : 인간의 언어는 다양한 사투리, 강세, 신조어, 작문 스카일로 인해 매우 가변적이다. 알고리즘은 이러한 가변성을 처리할 수 있어야 하며 사용 중인 언어를 이해할 수 있어야 한다.
- 구조 : 인간의 언어는 문장이나 구의 의미를 이해할 때 구문을 파악하여 의미를 해석한다. 알고리즘은 문장의 구조와 문법적 요소를 이해하여 의미를 추론하거나 분석할 수 있어야 한다.

위와 같은 문제를 이해하고 구분할 수 있는 모델을 만들려면 우선 말물치(corpus)를 일정한 단위인 토큰(token)으로 나눠야 한다. 이때 말뭉치는 뉴스 기사, 사용자 리뷰, 저널이나 컬럼 등에서 목적에 따라 구축되는 텍스트 데이터이며, 토큰은 개별 단어나 문장 부호와 같은 텍스트이다. 말뭉치를 토큰으로 나누는 목적은 컴퓨터가 자연어를 이해할 수 있게 나누는 과정이며 이러한 과정을 **토큰화**라고 한다. 토큰화를 위해 토크나이저를 사용하는데 이는 텍스트 문자열을 토큰으로 나누는 알고리즘 또는 소프트웨어를 의미한다. 이 알고리즘은 텍스트에서 발생하는 다양한 단어와 구문을 식별하고 분석할 수 있게 하므로 언어 모델링 또는 기계 번역과 같은 다양한 자연어 처리 작업에서 사용된다.

토큰을 나누는 기준은 구축하려는 시스템이나 주어진 상황에 따라 다르며 이에 따라 시스템의 성능이나 처리 결과가 달라진다. 토크나이저를 구축하는 방법은 다음과 같다.

- 공백 분할 : 텍스트를 공백 단위로 분리해 개별 단어로 토큰화한다.
- 정규표현식 적용 : 정규표현식으로 특정 패턴을 식별해 텍스트를 분할한다.
- 어휘 사전 적용 : 사전에 정의된 단어 집합을 토큰으로 사용한다.

- 머신러닝 활용 : 데이터세트를 기반으로 토큰화하는 방법을 학습한 머신러닝을 적용한다.

이 방법 중 어휘 사전은 사전에 정의된 단어를 활용해 토크나이저를 구축하는 방법인데, 직접 어휘 사전을 구축하기 때문에 없는 단어나 토큰이 존재할 수 있다. 이러한 토큰을 OOV(Out of Vocab)라 한다. 이러한 OOV 문제뿐만 아니라 크기 역시 고려해야 한다.

단어 및 글자 토큰화

토큰화를 할 때 입력된 텍스트 데이터를 단어나 글자 단위로 나누는지에 따라서 단어 토크화와 글자 토큰화로 나눈다.이러한 기법을 통해 각각의 토큰은 의미를 갖는 최소 단위로 분해된다.

1. 단위 토큰화 : 띄어쓰기, 문장 부호, 대소문자 등의 특정 구분자를 활용해 토큰화가 수행된다. 문자열 데이터 형태는 split 메서드를 이용하여 쉽게 토큰화하여 리스트 데이터로 나눠준다.

```
# 5.1 단어 토큰화
review = "현실과 구분 불가능한 cg. 시각적 즐거움은 최고! 더불어 ost는 더더욱 최고!!"
tokenized = review.split()
print(tokenized)

출력 : ['현실과', '구분', '불가능한', 'cg.', '시각적', '즐거움은', '최고!', '더불어', 'ost는', '더더욱', '최고!!']
```

그러나, 단어 토큰화는 한국어 접사, 문장 부호, 오타 혹은 띄어쓰기 오류 등에 취약하다

2. 글자 토큰화 : 글자 단위로 문장을 나누는 방식으로, 비교적 작은 단어 사전을 구축할 수 있다는 장점이 있다. 글자 토큰화는 언어 모델링과 같은 시퀀스 예측 작업에서 활용된다. list 형태로 변환하여 수행한다.

```
#5.2 글자 토큰화
review = "현실과 구분 불가능한 cg. 시각적 즐거움은 최고! 더불어 ost는 더더욱 최고!!"
tokenized = list(review)
print(tokenized)

출력 : ['현', '실', '과', ' ', '구', '분', ' ', '불', '가', '능', '한', ' ', 'c', 'g', '.', ' ', '시', '각', '적', ' ', '즐', '거', '움', '은', ' ', '최', '고', '!', ' ', '더',
```

```
'불', '어', ' ', 'o', 's', 't', '는', ' ', '더', '더',
'욱', ' ', '최', '고', '!', '!']
```

3. 자소 단위 토큰화

한편 영어의 경우 글자 토큰화를 진행하면 각 알파벳으로 나뉘는데 한글의 경우 여러 자음과 모음으로 조합으로 이루어져있기 때문에, 자소 단위로 나눠서 자소 단위 토큰화를 수행한다. 책에서는 자모(jamo) 라이브러리를 사용한다.(pip install jamo로 자모 라이브러리 설치).

자모 변환 함수는 입력된 한글 문자열을 유니코드 U+1100~U+11FE 사이으이 조합형 한글 자모로 변환하는 함수다. 컴퓨터가 한글을 인코딩하는 방식은 크게 조합형과 완성형으로 나눌 수 있다ㅏ.

- 조합형 : 글자를 자모 단위로 나눠 인코딩한 뒤 이를 조합해 한글을 표현한다.
- 완성형 : 조합된 글자 자체에 값을 부여해 인코딩하는 방식이다.

jamo의 h2j함수는 완성형으로 입력된 한글을 조합형 한글로 변환하고, j2hcj 함수는 조합형 한글 문자열을 자소 단위로 나눠 반환한다.

```
# 5.3 자소 단위 토큰화
from jamo import h2j, j2hcj

decomposed = j2hcj(h2j(review))
tokenized = list(decomposed)
print(tokenized)

출력 :  'ㅎ', 'ㅕ', 'ㄴ', 'ㅅ', 'ㅣ', 'ㄹ', 'ㄱ', 'ㅘ', ' ',
'ㄱ', 'ㅜ', 'ㅂ', 'ㅜ', 'ㄴ', ' ', 'ㅂ', 'ㅜ', 'ㄹ', 'ㄱ',
'ㅏ', 'ㄴ', 'ㅡ', 'ㅇ', 'ㅎ', 'ㅏ', 'ㄴ', ' ', 'c', 'g',
'.', ' ', 'ㅅ', 'ㅣ', 'ㄱ', 'ㅏ', 'ㄱ', 'ㅈ', 'ㅓ', 'ㄱ', '
', 'ㅈ', 'ㅡ', 'ㄹ', 'ㄱ', 'ㅓ', 'ㅇ', 'ㅜ', 'ㅁ', 'ㅇ',
'ㅡ', 'ㄴ', ' ', 'ㅊ', 'ㅚ', 'ㄱ', 'ㅗ', '!', ' ', 'ㄷ',
'ㅓ', 'ㅂ', 'ㅜ', 'ㄹ', 'ㅇ', 'ㅓ', ' ', 'o', 's', 't',
'ㄴ', 'ㅡ', 'ㄴ', ' ', 'ㄷ', 'ㅓ', 'ㄷ', 'ㅓ', 'ㅇ', 'ㅜ',
'ㄱ', ' ', 'ㅊ', 'ㅚ', 'ㄱ', 'ㅗ', '!', '!']
```

이러한 글자코튼화는 단어 토큰화보다 비교적 적은 크기의 단어 사전을 구축할 수 있고, '도', '는'과 같은 접사와 문장 부호의 의미를 학습할 수 있고, 작은 크기의 단어 사전으로도 OOV를 획기적으로 줄일 수 있다. 하지만 개별 토큰은 아무런 의미가 없으므로 자연

어 모델이 각 토큰의 의미를 조합해 결과를 도출해야 한다. 또한 모델 입력 시퀀스의 길이가 길어질수록 연산량이 증가한다는 단점이 있다.(예를 단어 단어의 경우 입력 시퀀스의 길이 13, 글자는 53, 자소 단위 토큰화의 경우 101까지 늘어남)

형태소 토큰화

텍스트를 형태소 단위로 나누는 토큰화 방법으로 언어의 문법과 구조를 고려해 단어를 분리하고 이를 의미 있는 단위로 분류한다. 이는 특히 한국어와 같이 교착어인 언어에서 중요하게 수행된다. 형태소 어휘 사전은 자연어 처리에서 사용되는 단어의 집합인 어휘 사전 중에서도 각 단어의 형태소 정보를 포함하는 사전을 말한다. 단어가 어떤 형태소들의 조합으로 이루어져 있는지에 대한 정보를 담고 있어, 형태소 분석 작업에서 매우 중요한 역할을 한다.

이때 각 형태소가 어떤 품사에 속하는지와 해당 품사의 뜻 등의 정보도 함께 제공된다. 텍스트 데이터를 형태소 분석하여 각 형태소에 해당하는 품사(Part of Speech, POS)를 태깅하는 작업을 품사 태깅이라고 한다. 이를 통해 자연어 처리 분야에서 문맥을 고려할 수 있어 더욱 정확한 분석이 가능해 진다. 형태소 토큰화를 실습할 수 있는 라이브러리는 KoNLPy, NLTK, spaCy가 있다.

- KoNLPy

한국어 자연어 처리를 위해 개발된 라이브러리로 명사 추출, 형태소 분석, 품사 태깅 등의 기능을 제공한다. KoNLPy는 자바 언어 기반의 한국어 형태소 분석기로 자바 개발 키트 기반으로 개발됐으므로 자바 개발 키트가 설치돼 있어야 한다.

KoNLPy는 Okt(Open Korea Text), 꼬꼬마(Komoran), 한나눔(Hannanum), 메캅(Mecab) 등의 다양한 형태소 분석기를 지원한다.

- Okt에서 지원하는 대표적인 메서드는 명사추출(okt.nouns), 구문 추출(okt.phrases), 형태소 추출(okt.morphs), 품사 태깅(okt.pos)이다.

```
# 5.4 Okt 토큰화
from konlpy.tag import Okt

okt = Okt()

sentence = "무엇이든 상상할 수 있는 사람은 무엇이든 만들어낼 수 있다."

nouns = okt.nouns(sentence)
phrases = okt.phrases(sentence)
morphs = okt.morphs(sentence)
pos = okt.pos(sentence)
```

```
print("nouns : ", nouns)
print("phrases : ", phrases)
print("morphs : ", morphs)
print("pos : ", pos)

출력 :
nouns :  ['무엇', '상상', '수', '사람', '무엇', '낼', '수']
phrases :  ['무엇', '상상', '상상할 수', '상상할 수 있는 사
람', '사람']
morphs :  ['무엇', '이든', '상상', '할', '수', '있는', '사
람', '은', '무엇', '이든', '만들어', '낼', '수', '있다',
'.']
pos :  [('무엇', 'Noun'), ('이든', 'Josa'), ('상상', 'Nou
n'), ('할', 'Verb'), ('수', 'Noun'), ('있는', 'Adjectiv
e'), ('사람', 'Noun'), ('은', 'Josa'), ('무엇', 'Noun'),
('이든', 'Josa'), ('만들어', 'Verb'), ('낼', 'Noun'),
('수', 'Noun'), ('있다', 'Adjective'), ('.', 'Punctuati
on')]
```

- 꼬꼬마는 구문 추출 기능은 지원하지 않지만 명사 추출(kkma.sentences) 기능을 제공한다.

```
# 5.5 꼬꼬마 토큰화
from konlpy.tag import Kkma

kkma = Kkma()

sentence = "무엇이든 상상할 수 있는 사람은 무엇이든 만들어낼 수
있다."

nouns = kkma.nouns(sentence)
sentences = kkma.sentences(sentence)
morphs = kkma.morphs(sentence)
pos = kkma.pos(sentence)

print("nouns : ", nouns)
print("phrases : ", phrases)
```

```
print("morphs : ", morphs)
print("pos : ", pos)

출력:
nouns :  ['무엇', '상상', '수', '사람', '무엇']
phrases :  ['무엇', '상상', '상상할 수', '상상할 수 있는 사
람', '사람']
morphs :  ['무엇', '이', '든', '상상', '하', 'ㄹ', '수',
'있', '는', '사람', '은', '무엇', '이', '든', '만들', '어',
'내', 'ㄹ', '수', '있', '다', '.']
pos :  [('무엇', 'NNG'), ('이', 'VCP'), ('든', 'ECE'),
('상상', 'NNG'), ('하', 'XSV'), ('ㄹ', 'ETD'), ('수', 'N
NB'), ('있', 'VV'), ('는', 'ETD'), ('사람', 'NNG'),
('은', 'JX'), ('무엇', 'NP'), ('이', 'VCP'), ('든', 'EC
E'), ('만들', 'VV'), ('어', 'ECD'), ('내', 'VXV'),
('ㄹ', 'ETD'), ('수', 'NNB'), ('있', 'VV'), ('다', 'EF
N'), ('.', 'SF')]
```

- 한편 둘을 비교하면 동일한 메서드라도 형태소 분석기의 특징에 따라 명사 추출 혹은 품사 태깅 등 다른 결과가 나타난다. 그러므로 시스템의 목적과 환경에 적절한 형태소 분석기를 사용해야 한다.

- NLTK

주로 영어 자연어 처리를 위해 개발되었지만, 네덜란드어, 프랑스어, 독일어 등과 같은 다양ㅇ한 언어의 자연어 처리를 위한 데이터와 모델을 제공한다. 이 책에서는 Punkt 모델과 Averaged Perception Tagger 모델을 활용해 토큰화 및 품사 태깅 작업을 수행하겠다. 두 모델 모두 트리뱅크라는 대규모의 영어 말뭉치를 기반으로 학습됐다. Punkt 모델은 통계 기반 모델이며, Averaged Perception Tagger는 퍼셉트론을 기반으로 품사 태깅을 수행한다.

```
import nltk

nltk.download("punkt")
nltk.download("averaged_perception_tagger")
```

위와 같은 코드를 모델을 다운로드 받을 수 있다.

영문 토큰화는 punkt 모델을 기반으로 토큰화한다. 단어 토크나이저(word_tokenize)는 공백을 기준으로, 문장 토크나이저(sent_tokenize)는 구두점을 기준으로 문장을 분

리해 리스트로 반환한다. 또한 품사 태깅은 Averaged Perceptron Tagger 모델로 35개의 품사에 대해서 태깅힐 수 있다.

```
# 5.7-8 영문 토큰화 및 품사 태깅
from nltk import tokenize
from nltk import tag

sentence = "Those who can imagine anything, can create impossible."

word_tokens = tokenize.word_tokenize(sentence)
sent_tokens = tokenize.sent_tokenize(sentence)
pos = tag.pos_tag(word_tokens)

print(word_tokens)
print(sent_tokens)
print(pos)

출력 :
['Those', 'who', 'can', 'imagine', 'anything', ',', 'can', 'create', 'impossible', '.']
['Those who can imagine anything, can create impossible.']
[('Those', 'DT'), ('who', 'WP'), ('can', 'MD'), ('imagine', 'VB'), ('anything', 'NN'), (',', ','), ('can', 'MD'), ('create', 'VB'), ('impossible', 'JJ'), ('.', '.')]
```

- SpaCy

사이썬 기반으로 효율적인 처리 속도와 높은 정확도를 제공한다. GPU 가속을 비롯해 24개 이상의 언어로 사전 학습된 모델을 제공한다. 사사전 학습된 모델을 기반으로 처리하기 때문에 spacy.load를 통해 모델을 설정할 수 있다.

```
# 5.9 spaCy 품사 태깅
import spacy

nlp = spacy.load("en_core_web_sm")
sentence = "Those who can imagine anything can ccreate the impossible"
```

```
doc = nlp(sentence)

for token in doc:
  print(f"[{token.pos_:5}-{token.tag_:3}]:{token.text}")


출력 :
[PRON -DT ]:Those
[PRON -WP ]:who
[AUX  -MD ]:can
[VERB -VB ]:imagine
[PRON -NN ]:anything
[AUX  -MD ]:can
[VERB -VB ]:ccreate
[DET  -DT ]:the
[ADJ  -JJ ]:impossible
```

spacy는 객체 지향적으로 구현돼 처리한 결과를 doc 객체에 저장한다. doc 객체는 다시 여러 token 객체로 이루어져 있으며, 이 token에 대한 정보를 기반으로 다양한 자연어 처리 작업을 수행한다. nlp 인스턴스에 문장을 입력하면 doc 객체가 반환되며 toekn 객체의 tag_나 pos_와 같은 속성에 접든해 값을 확인할 수 있다.

token 객체에는 기본 품사 속성(pos_), 세분화 품사 속성(tag_), 원본 텍스트 데이터(text), 토큰 사이의 공백을 포함하는 텍스트 데이터(text_with_ws), 벡터(vector), 벡터 노름(vector_norm) 등의 속성이 포함돼 있다.

하위 단어 토큰화

형태소 단위의 토큰화를 통해 효과적으로 자연어 처리를 할 수 있지만 언어는 시간이 지남에 따라 변화하고 새로운 단어나 표현이 등장하며 더 이상 사용되지 않는 단어나 표현도 생긴다. 예를 들어 다음과 같은 문장을 기존의 형태소 분석기로 토큰화하면 다음과 같은 결과가 나온다

- 원문: 시보리도 짱짱해고 허리도 어벙하지안구 죠하효
- 결과: ['시', '보리', '도', '짱짱해고', '허리', '도', '어', '벙하지안구', '죠', '하', '효']

- 원문: 진짜 멋있네요! 돈쭐날만 하네요.
- 결과: ['진짜', '멋있네요', '!', '돈쭐날', '만', '하네요', '.']

이처럼 형태소 분석기는 전문용어나 고유어, 신조어가 많은 데이터를 처리할 때 약점을 보인다. 즉 형태소 분석기는 모르는 단어를 적절한 단위로 나누는 것에취약하며, 이는 잠재적으로 어휘 사전의 크기를 크게 만들고 OOV에 대응하기 어렵게 만든다. 이를 해결하기 위한 방법 중 하나로 **하위 단어 토큰화(Subword Tokenization)**가 있다. 이는 하나의 단어가 빈번하게 사용되는 하위 단어의 조합으로 나누어 토큰화를 하는 방법이다. 예를 들어 reinforcement라는 단어에 대해 rein, force, ment 등으로 나누어 처리할 수 있다. 이를 적용하면 단어의 길이를 줄일 수 있어 처리 속도가 빨라질 뿐만 아니라, OOV 문제, 신조어, 은어, 고유어 등으로 인한 문제를 완화할 수 있다. 하위 토큰화 방법으로는 바이트 페어 인코딩, 워드피스, 유니그램 모델 등이 있으며, 책에서는 바이트 페어 인코딩과 워드피스 알고리즘을 다룬다.

- 바이트 페어 인코딩(Byte pair encoding)

다이그램 코딩이라고도 하며 하위 단어 토큰화의 한 종류다. 텍스트 데이터에서 사장 빈번하게 등장하는 글자 쌍의 조합을 부화하하는 압축 알고리즘으로 초기에는 데이터 압축을 위해 개발됐으나, 자연어 처리 분야에서 하위 단어 토큰화를 위한 방법으로 사용된다. 이 알고리즘은 연속된 글자 쌍이 더 이상 나타나지 않거나 정해진 어휘 사전 크기에 도달할 때까지 조합 탐지와 부호화를 반복하여 이 과정에서 자주 등장하는 단어는 하나의 토큰으로 토큰화되고, 덜 등장하는 단어는 여러 토큰의 조합으로 표현된다. 더 이상 치환할 수 있는 글자 쌍이 존재하지 않을 때까지 데이터를 압축하며, 자주 등장하는 글자 쌍을 찾아 치환하는 대신 어휘 사전에 추가한다.

- 원문: abracadabra
- Step #1: AracadAra
- Step #2: ABcadAB
- Step #3: CcadC

말뭉치에서 BPE를 적용하기 위해 말뭉치에서 각 단어가 등장한 빈도를 계산해 다음과 같은 빈도 사전과 어휘 사전을 만들었다고 가정해보자.

- 빈도 사전: ('low', 5), ('lower', 2), ('newest', 6), ('widest', 3)
- 어휘 사전: ['low', 'lower', 'newest', 'widest']

빈도사전 내의 모든 단어를 글자 단위로 나누고 가장 자주 등장한 글자 쌍을 찾아 어휘 사전에 가장 자주 등장한 글자 쌍을 추가한다.

- 빈도 사전: ('low', 5), ('low', 'e', 'r', 2), ('n', 'e', 'w', 'est', 6), ('w', 'i', 'd', 'est', 3)
- 어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w', 'es', 'est', 'lo', 'low', 'ne', 'new', 'newest', 'wi', 'wid', 'widest']

이를 계속 반복하면 다음과 같은 빈도 사전과 어휘 사전이 생성된다.

- 빈도 사전: ('low', 5), ('low', 'e', 'r', 2), ('n', 'e', 'w', 'est', 6), ('w', 'i', 'd', 'est', 3)
- 어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w', 'es', 'est', 'lo', 'low', 'ne', 'new', 'newest', 'wi', 'wid', 'widest']

즉 BPE 알고리즘을 사용한면 newer, wider, lowest와 같이 기존 말뭉치에 등장하지 않았던 단어가 입력되더라도 OOV로 처리되지 않고 기존 어휘 사전을 참고해 new e r, wid e r, low est 와 같이 토큰화할 수 있다.

- 센턴스피스(sentencepiece)와 코포라(Korpora)

센텐스피스 라이브러리는 구글에서 개발한 오픈 소스 하위 단어 토크나이저 라이브러리다.BPE와 유사한 알고리즘을 사용해 입력 데이터를 토큰화하고 단어 사전을 생성한다. 또한 워드피스, 유니코드 기반의 다양한 알고리즘을 지원하며 사용자가 직접 설정할 수 있는 하이퍼파라미터들을 제공해 세밀한 토크나이징 기능을 제공한다. 코포라 라이브러리는 국립 국어원이나 AI Hub에서 제공하는 말뭉치 데이터를 쉽게 사용할 수 있게 제공하는 오픈 소스 라이브러리이다.

```
# 5.10 청와대 청원 데이터 다운로드
from Korpora import Korpora

corpus = Korpora.load("korean_petitions")
dataset = corpus.train
petition = dataset[0]

print("청원 시작일:", petition.begin)
print("청원 종료일:", petition.end)
print("청원 동의 수:", petition.num_agree)
print("청원 범주:", petition.category)
print("청원 제목:", petition.title)
print("청원 본문:", petition.text[:30])

출력
청원 시작일: 2017-08-25
청원 종료일: 2017-09-24
```

```
청원 동의 수: 88
청원 범주: 육아/교육
청원 제목: 학교는 인력센터, 취업센터가 아닙니다. 정말 간곡히 부탁드
립니다.
청원 본문: 안녕하세요. 현재 사대, 교대 등 교원양성학교들의 예비
```

corpus는 train 속성으로 학습 데이터를 가지고 올 수 있다. 첫 번째 청원 데이터는 dataset[0]d으로 불러올 수 있으며, petition에서 청원 시작일, 동의수, 제목 등의 정보가 포함되어 있다.

```
# 5.11 학습데이터세트 생성
corpus = Korpora.load("korean_petitions")
petitions = corpus.get_all_texts()
with open("../datasets/corpus.txt", "w", encoding="utf
-8") as f:
  for petition in petitions:
    f.write(petition + "\n")
```

센텐스피스의 주요한 하이퍼파라미터

표 5.5 SentencePieceTrainer 매개변수

매개변수	의미
input	말뭉치 텍스트 파일의 경로
model_prefix	모델 파일 이름
vocab_size	어휘 사전 크기
character_coverage	말뭉치 내에 존재하는 글자 중 토크나이저가 다룰 수 있는 글자의 비율
model_type	토크나이저 알고리즘 ▪ unigram ▪ bpe ▪ char ▪ word
max_sentence_length	최대 문장 길이
unk_id	어휘 사전에 없는 OOV를 의미하는 unk 토큰의 id (기본값: 0)
bos_id	문장이 시작되는 지점을 의미하는 bos 토큰의 id (기본값: 1)
eos_id	문장이 끝나는 지점을 의미하는 eos 토큰의 id (기본값: 2)

센텐스피스 라이브러리는 띄어쓰기나 공백도 특수문자로 취급해 토큰화 과정에서 '_'으로 공백을 표현한다. 가령 "Hello World"라는 문장은 "Hello_World"로 표현되며 토큰화하면 "_Hello _Wor +ld"로 토큰화된다.

encode_as_pieces 메서드는 문장을 토큰화하며, encode_as_ids 메서드는 토큰을 정수로 인코딩해 제공한다. 이 정수 데이터는 어휘 사전의 토큰에 매핑된 ID 값을 의미한다. 이 ID 값을 활용해 자연어 처리 모델을 구축한다. 이러한 정수는 decode_ids 메서드나 deocde_pieces 메서드를 통해 문자열 데이터로 변환할 수 있다.

get_piece_size 메서드는 센텐스피스 모델에서 생성된 하위 단어의 개수를 반환하며, id_to_piece 메서드는 정숫값을 하위 단어로 변환하는 메서드다. 그러므로 하위 단어의 개수만큼 반복해 하위 단어 딕셔너리를 구상한다.

토큰 딕셔너리를 출력한다면 <unk>, <s>, </s>가 존재하는데 <unk> 토큰은 unkwon의 약자로 OOV 발생시 매핑되는 토큰이며, <s></s>는 문장의 시작 지점과 종료 지점을 표시하는 토큰이다.

- 워드피스

워드피스 토크나이저는 BPE 토크나이저와 유사한 방법으로 학습되지만, 빈도 기반이 아닌 확률 기반으로 글자 쌍을 병합한다. 즉 모델이 새로운 하위 단어를 생성할 때 이전 하위 단어와 함께 나타날 때 확률을 계산해 가장 높은 확률을 가진 하위 단어를 선택한다. 이렇게 선택된 하위 단어는 이후에 더 높은 확률로 선택될 가능성이 높으며, 이를 통해 모델이 좀 더 정확한 하위 단어로 분리할 수 있다. 각 글자쌍에 대한 점수는 다음과 같다.

$$score = \frac{f(x,y)}{f(x),f(y)}$$

f는 빈도, x와 y는 병합하려는 하위 단어를 의미한다. 그러므로 f(x, y)는 x,y가 조합된 글자 쌍의 빈도를 의미한다. 즉 xy 글자 쌍의 빈도가 된다. 그러므로 score는x와 y를 병합하는 것이 적절한지를 판단하기 위한 점수가 된다.

- 빈도 사전: ('l', 'o', 'w', 5), ('l', 'o', 'w', 'e', 'r', 2), ('n', 'e', 'w', 'e', 's', 't', 6), ('w', 'i', 'd', 'e', 's', 't', 3)
- 어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w']

이전의 예시를 통해 다시 본다면 가장 빈번하게 등장한 쌍은 e와 s이다. 하지만 e는 17번, s는 9번 등장하므로 점수는 9/17*9로 0.06이 된다. 이를 반복해 연속된 글자 쌍이 더 이상 나타나지 않거나 정해진 어휘 사전 크기에 도달할 때까지 학습한다.

- 토크나이저스

토크나이저스 라이브러리의 워드피스 API를 이용하면 쉽고 빠르게 토크나이저를 구현하고 학습할 수 있다. 토크나이저스 라이브러리는 정규하ㅗ와 사전 토큰화를 제공한다.

정규화는 일관된 형식으로 텍스트를 표준화하고 모호한 경우를 방지하기 위해 일부 문자를 대체하거나 제거하는 등의 작업을 수행한다. 불필요한 공백 제거, 대소문자 변환, 유니코드 정규화, 구두점 처리, 특수문자 처리 등을 제공한다. normalizers 모듈에 포함된 클래스를 불러와 시퀀스 형식으로 인스턴스를 전달한다. 예제에ㅐ서 적용한 정규화방식은 NFD 유니코드 정규화(NFD), 소문자 변환(Lowercase)을 사용한다.

표 5.6 정규화 클래스

이름	설명
NFD()	NFD 유니코드 정규화
NFKD()	NFKD 유니코드 정규화
NFC()	NFC 유니코드 정규화
NFKC()	NFKC 유니코드 정규화
Lowercase()	입력 문장의 모든 대문자를 소문자로 변환
Strip()	입력 문장의 양 끝에 있는 공백 제거
StripAccents()	모든 엑센트 기호 제거
Replace()	문자 혹은 정규 표현식을 기반으로 입력 문장 변환
BertNormalizer()	BERT 모델에 사용된 정규화 적용
Sequence()	여러 개의 정규화 모듈을 병합해 순서대로 수행

사전 토큰화는 입력 문장을 토큰화하기 전에 단어와 같은 작은 단위로 나누는 기능을 제공한다. 공백 혹은 구두점을 기준으로 이력 문장을 나눠 텍스트 데이터를 효율적으로 처리학고 모델의 성능을 향상시킬 수 있다. pre_tokenizers 모듈에 ㅍ포함된 클래스를 불러와 적용한다. 예저에서 적용한 사전 토큰화 방식은 공백과 구두점을 기준으로 분리한다.

표 5.7 사전 토큰화 클래스

클래스	설명
`Sequence()`	여러 개의 사전 토큰화 모듈을 병합하여 순서대로 수행
`ByteLevel()`	OpenAI에서 GPT-2를 학습할 때 사용한 방법으로 문자열을 그래픽 문자열로 매핑하고 공백을 기준으로 나눈다.
`CharDelimiterSplit()`	문자 기준으로 문자열을 나눈다.
`Digits()`	모든 형태의 숫자 표현을 기준으로 문자열을 나눈다.
`Punctuation()`	구두점을 기준으로 입력 문자열을 나눈다.

`Metaspace(`
`  replacement="_"`
`)`
	`Whitespace` 기준으로 사전 토큰화를 진행하고, `replacement`로 `Whitespace`를 치환한다.
	▪ 기본값: _(U+2581)

`Whitespace()`	단어 경계(공백과 구두점)를 기준으로 사전 토큰화를 진행한다. ▪ `\w+|[^\w\s]+` 정규 표현식을 적용한다.
`WhitespaceSplit()`	공백 문자를 기준으로 입력 문자열을 나눈다.

`Split(`
`  pattern,`
`  behavior`
`)`
	정규표현식(pattern)과 동작(behavior)을 기준으로 문자열을 나눈다.
	▪ behavior
	– `removed`: 삭제
	– `isolated`: 개별 토큰 처리
	– `merged_with_previous`: 이전 문자열과 결합해 토큰 처리
	– `merged_with_next`: 다음 문자열과 결합해 토큰 처리
	– `contiguous`: 다음 문자열이 토큰화되지 않아도 결합해 토큰 처리

워드피스 토큰화를 위해 모델 결과가 저장된 파일을 불러와 Tokenizer 객체를 생성하고, WordPieceDecoder()를 사용해 Tokenizer의 디코더를 워드피스 디코더로 설정한다.

▼ 6장 : 임베딩

컴퓨터는 텍스트 자체를 이해할 수 없으므로 토큰화뿐만 아니라 텍스트를 숫자로 변환하는 텍스트 벡터화 과정이 필요하다. 텍스트 벡터화란 텍스를 숫자로 변환하는 과정을 의미한다. 기초적인 벡터화로는 원-핫 인코딩(One-Hot Encoding), 빈도 벡터화(Count Vectorization) 등이 있다.

원-핫 디코딩이란 문서에 등장하는 각 단어를 고유한 색인 값으로 매핑한 후 핵당 색인 위치를 1로ㅗ 표시하고 나머지 위치는 모두 0으로 표시하는 방식이다. 예를 들어 'I like apples' 문장과 'I like bananas' 문장을 띄어쓰기 기준으로 토큰화하고 단어를 고유한 색인 값으로 매핑하면 다음과 같다.

표 6.1 원-핫 인코딩 매핑 테이블

색인	0	1	2	3
토큰	I	like	apples	bananas

이를 바탕으로 각 문장을 원-핫 인코딩하면 다음과 같이 표현된다.



빈도벡터화는 문서에 서 단어의 빈도수를 세어 해당 단어릐 빈도를 벡터로 표현하는 방식이다. 예를 들어 apples라는 단어가 총 4번 등장한다면, 해당 단어에 대한 벡터값은 4가 된다.

원-핫 인코딩 방식은 같은 단어가 여러 번 등장하더라고 1과 0으로 표현하지만, 빈도 벡터화는 해당 단어의 빈도로 표시한다. 이러한 방법은 단어나 문장을 벡터 형태로 변환하기 쉽고 간단하다는 장점이 있지만 벡터의 희소성(Sparsity)이 크다는 단점이 있다. 또한 텍스트의 벡터가 입력 텍스트의 의미를 내포하고 있지 않으므로 두 문장이 의미적으로 유사하다고 해도 벡터가 유사하게 나타나지 않을 수 있다.

이러한 문제를 해결하기 위해 워드 투 벡터(Word2Vec)나 패스트 텍스트(fastText) 등과 같이 단어의 의미를 학습해 표현하는 **워드 임베딩** 기법을 사용한다. 워드 임베딩 기법은 단어를 고정된 길이의 실수 벡터로 표현하는 방법으로, 단어의 의미를 벡터 공간에서 다른 단어와의 상대적 위치로 표현해 단어 간의 관계를 추론한다. 워드 임베딩은 고정된 임베딩을 학습하기 떄문에 다의어나 문맥 정보를 다루기 어렵다는 단점이 있어 인공신경망을 활용해 **동적 임베딩** 기법을 사용한다.

언어 모델

언어 모델이란 입력 된 문장으로 각 문장을 생성할 수 있는 확률을 계산하는 모델을 의미한다. 이를 위해 주어진 문장을 바탕으로 문맥을 이해하고, 문장 구성에 대한 예측을 수행한다. 그러나 '안녕하세요'라는 문장 뒤에 어떤 문장이 어떤 확률로 등장할지 계산하는 것은 어려운 일이다. 이러한 문제ㅐ를 해결하기 위해 문장 전체를 예측하는 방법ㅇ 대신 하나의 토큰 단위로 예측하는 방법인 자기회귀 언어 모델이 고안됐다.

- 자기회귀 언어 모델(Autoregressive Language Model)

입력된 문장들의 조건부 확률을 이용해 다음에 올 단어를 예측한다. 즉 언어 모델에서 조건부 확률은 이전 단어들의 시퀀스가 주어졌을 때 다음 단어의 확률을 계산하는 것을 의미한다. 이를 위해 이전에 등장한 모든 토큼의 정보를 고려하여, 문장의 문맥 정보를 파악하여 다음 단어를 생성한다. 다음 단어는 다시 이전 단어를 기반으로 예측이 이루어지며 이 과정이 반복된다.

수식 6.1 언어 모델의 조건부 확률

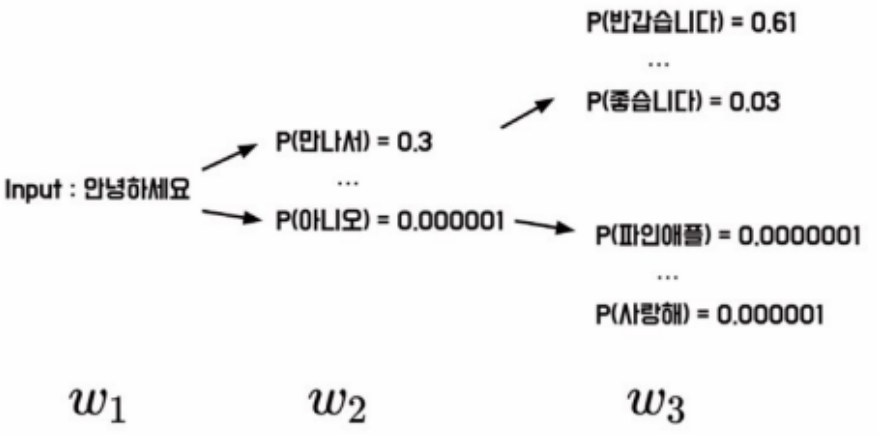
$$P(w_t|w_1, w_2, \cdots, w_{t-1}) = \frac{P(w_1, w_2, \cdots, w_t)}{P(w_1, w_2, \cdots, w_{t-1})}$$

여기에 조건부 확률의 연쇄법칙으르 적용한다면 다음과 같다.

수식 6.2 조건부 확률의 연쇄법칙을 적용한 언어 모델의 조건부 확률

$$P(w_t|w_1, w_2, \cdots, w_{t-1}) = P(w_1)P(w_2|w_1)\cdots P(w_t|w_1, w_2, \cdots, w_{t-1})$$

문장 전체의 확률은 첫 번째 단어의 확률 P(w_i)과 단어가 이전 단어들의 조건부 확률에 따라 발생할 확률의 곱으로 나타낼 수 있다. 이를 통해 언어 모델은 문장 전체의 확률을 계산하고, 그것을 이용해 다음 단어를 예측한다.



자기 회귀 언어 모델에서는 각 시점에서 다음과 올 토큰을 예측하는 것이 중요하다. 이를 위해 입력 토큰 w_i을 바탕으로 모델은 다음 토큰 w_2가 등장할 확률 P(w_2|w_1)을 예측한다. 그다음 모델의 출력값 w_2가 다시 입력값이 되어, 모델은 호가률 P(w_3\w_1, w_1)를 예측한다.

이러한 방식으로 모델의 출력값이 모델의 입력값으로 사용되는 특징 때문에 자기회귀라는 이름이 붙었
다. 자기회귀 언어 모델은 시점별로 다음에 올 토큰을 예측하는 것이므로, 토큰 분류 문제로 정의할 수
있다. 자기회귀 모델은 이전에 등장한 모든 토큰의 정보를 활용해 입력된 문장의 문맥 정보를 파악하고 다음 토큰을 예측한다. 그러므로 언어 모델은 문장 전체의 확률을 계산하고, 이를 이용해 다음 단어를 예측한다.

- 통계적 언어 모델

통계적 언어 모델은 언어의 통계적 구조를 이용해 문장이나 단어의 시퀀스를 생성하거나 분석한다. 시퀀스에 대한 확률 분포를 추정해 문장의 문맥을 파악해 다음에 등장할 단어의 확률을 예측한다. 일반적으로 통계적 언어 모델은 마르코프 체인을 이용해 구현된다. 마르코프 체인은 빈도 기반의 조건부 확률 모델 중 하나로 이전 상태와 현재 상태 간의 전이 확률을 이용해 다음 상태를 예측한다.

예를 들어 말뭉치에 '안녕하세요'라는 문장이 1000번 등장하고 이어서 '안녕하세요 만나서가 700번, '안녕하세요 반갑습니다'가 100번 정도 등장했다고 가정한다면 다음과 같다.

$$P(\text{만나서} \mid \text{안녕하세요}) = \frac{P(\text{안녕하세요 만나서})}{P(\text{안녕하세요})} = \frac{700}{1000}$$

$$P(\text{반갑습니다} \mid \text{안녕하세요}) = \frac{P(\text{안녕하세요 반갑습니다})}{P(\text{안녕하세요})} = \frac{100}{1000}$$

그러나 이 방법은 단어의 순서와 빈도에만 기초해 문장의 확률을 예측하므로 문맥을 제대로 파악하지 못하면 불완전하거나 부적절한 결과를 생성할 수 있다. 또한 한 번도 등장한 적이 없는 단어나 문장에 대해서는 정확한 확률을 예측하기가 어렵다. 이렇게 관측한 적이 없는 데이터를 예측하지 못하는 문제를 데이터 희소성 문제라고 한다.

하지만 통계적 언어 모델은 기존에 학습한 텍스트 데이터에서 패턴을 찾아 확률 분포를 생성하므로,
를 이용하여 새로운 문장을 생성할 수 있으며, 다양한 종류의 텍스트 데이터를 학습할 수 있다. 최근 연되는 자연어 처리 기법은 언어 모델을 활용해 가중치를 학습한다. 예를 들어 트랜스포머 모델에 기반한 언어 모델인 GPT나 BERT는 다음과 같은 문장 생성 기법을 이용해 모델을 사전 학습한다.

GPT

- Raw Sentence: GPT는 OpenAI에서 개발한 인과적 언어 모델입니다.
- Input: GPT는 OpenAI에서 개발한
- Prediction–1: 인과적
- Prediction–2: 언어 모델입니다.

BERT

- Raw Sentence: BERT는 Google에서 개발한 마스킹된 언어 모델입니다.
- Input: Bert는 [MASK]에서 개발한 [MASK] 언어 모델입니다.
- Prediction: Google, 마스킹된

N-gram

가장 기초적인 통계적 모델은 N-gram 모델이다. N-gram 모델은 텍스트에서 N개의 연속된 단어 시퀀스를 하나의 단위로 취급하여 특정 단어 시퀀스가 등장할 확률을 추정한다. 즉, 입력 텍스트를 하나의 토큰 단위로 분석하지 않고 N개의 토큰을 묶어서 분석한다는 뜻이다. N이 1일 때는 유니그램(Unigram), N이 2일때는 바이그램(Bigram), N이 3일 때는 트라이그램(Trigram), N이 4 이상이면 N-gram이라고 부른다.



그림 6.3 N이 1, 2, 3인 N-gram

N-gram 언어모델은 모든 토큰을 사용하지 않고 N-1개의 토큰만을 고려해 확률을 계산한ㄴ다. 따라서 t번째 토큰의 조건부 확률을 계산하는 수식은 다음과 같다.

$$P(w_t | w_{t-1}, w_{t-2}, \cdots, w_{t-N+1})$$

w_1은 예측하려는 단어, w_t-1부터 w_t-N+1까지는 예측에 사용되는 이전 단어들을 의미한다. 이전 단어들의 개수를 결정하는 N의 값을 조정하여 N-gram 모델의 성능을 조절할 수 있다.

N-gram은 작은 규모의 데이터세트에서 연속된 문자열 패턴을 분석하는 데 큰 효과를 보인다. 또한 관용적 표현 분석에도 활용된다.

TF-IDF(Term frequency invers doocument frequency)

텍스트 문서에서 특정 단어의 중요도를 계산하는 방법으로, 문서 내에서 단어의 중요도를 평가하는 데 사용되는 통계적인 가중치를 의미한다. w즉 TF-IDF는 Bow(Bag-of-Words)에 가중치를 부여하는 방법이다. BoW는 문서나 문장을 단어의 집합으로 표현하는 방법으로, 문서나 문장에 등장하는 단어의 중목을 허용해 빈도를 계산한다. 예를 들어, That movie is famous movie', 'I like that actor', I don't like that actor] 말뭉치를BoW로 벡터화하면 다음과 같다.

	I	like	this	movie	don't	famous	is
This movie is famous movie	0	0	1	2	0	1	1
I like this movie	1	1	1	1	0	0	0
I don't like this movie	1	1	1	1	1	0	0

BoW를 이용해 벡터화하는 경우 모든 단어는 동일한 가중치를 가지기 때문에 예를 들어 이를 활용해 영화 리뷰의 긍/부정 분류 모델을 만든다면 높은 성능을 얻기는 어렵다.

- 단어 빈도(Term Frequecny)

문서 내에서 특정 단어의 빈도수를 나타내는 값으로, 예를 드어 3개의 문서에서 movie라는 단어가 4번 등장한다면 해당 단어의 TF 값은 4가 된다. TF 값이 높을수록 해당 단어가 특정 문서에서 중요한 역할을 한다고 생각할 수도 있지만, 단어 자체가 특정 문서 내에서 자주 사용되는 단어이므로 전문 용어나 관용어로 간주할 수 있다. TF는 단순히 단어의 등장 빈도수를 계산하기 때문에 문서의 길이가 길어질수록 해당 단어의 TF 값도 높아질 수 있다.

$$TF(t,d) = count(t,d)$$

- 문서 빈도(DF)

한 단어가 얼마나 많은 문서에서 나타내는지를 의미한다. 특정 단어가 많은 문서에 나타나면 문서 집합에서 단어가 나타내는 횟수를 계산한다. DF는 단어가 몇 개의 문서에서 등장하는지 계산한다. DF 값이 높으면 특정 단어가 많은 문서에서 등장한다고 볼 수 있다. 그 단어는 일반적으로 널리 사용되며, 중요도가 낮을 수 있다. 반대로 DF 값이 낮다면 특정 단어가 적은 수의 문서에만 등장한다는 뜻이다. 그러므로 특정한 문맥에서만 사용되는 단어일 가능성이 있으며, 중요도가 높을 수 있다.

$$DF(t,D) = count(t \in d : d \in D)$$

- 역문서 빈도(IDF)

전체 문서 수를 문서 빈도로 나눈 다음에 로그를 취한 값을 말하며, 문서 내에서 특정 단어가 얼마나 중요한지 나타낸다. 따라서 문서 빈도가 높을수록 IDF 값이 작아진다.

$$IDF(t,D) = \log\left(\frac{count(D)}{1 + DF(t,D)}\right)$$

IDF는 분모의 DF 값에 1을 더한 값을 사용하는데 특정 단어가 한 번도 등장하지 않았다면 분모가 0이 되는 걸 방지하기 위함이며, 로그를 취하는 이유는 너무 적은 빈도수로 나오는 단어에 대해 값이 너무 크게 되는 걸 방지하기 위함이다.

최종적으로 TF-IDF는 문서빈도와 역문서 빈도를 곱한 값으로 수식은 다음과 같다.

$$TF - IDF(t,d,D) = TF(t,d) \times IDF(t,d)$$

문서 내에 단어가 자주 등장하지만, 전체 문서 내에 해당 단어가 적게 등장한다면 TF-IDF 값은 커진다. 그러므로 전체 문서에서 자주 등장할 확률이 높은 관사나 관용어 등의 가중치는 낮아진다.

```
# TF-IDF 클래스
from sklearn.feature_extraction.text import TfidfVectori
zer

tfidf_vectorizer = TfidfVectorizer(
    input="content",
    encoding="utf-8",
    lowercase=True,
    stop_words=None,
    ngram_range=(1, 1),
    max_df=1.0,
    min_df=1,
    vocabulary=None,
```

```
    smooth_idf=True
)
```

TF-IDF 클래스의 input은 입력될 데이터의 형태를 의미한다. 기본적으로 설정된 contenct는 문자열 데이터 혹은 바이트 형태의 입력값을 의미한다. 파일 객체를 사용한다면 file로 입력하며, 파일 경로를 사용하는 경우 filename으로 입력한다.

encoding은 바이트 혹은 파일을 입력값으로 받을 경우 사용할 텍스트 인코딩 값을 의미한다

lowercase는 입력받은 데이터를 소문자로 변환할지 여부를 말한다. 참값을 설정하면 모든 입력 테스트를 소문자로 변환한다.

stop_words(불용어)는 분석에 도움이 되지 않는 의미 없는 단어들을 의미하며, 입력받은 단어들은 단어 사전에 추가되지 않는다.

ngram_range는 사용할 N-gram의 범위로, (최솟값, 최댓값) 형태로 입력하며 (1, 1)은 유미그램, (1, 2)은 유니그램과 바이그램을 사용한다는 것을 의미한다.

max_df는 전체 문서 중 일정 횟수 이상 등장한 단어는 불용어로 처리한다. 정수를 입력하면 해당 등장 횟수를 초과해 등장하는 단어를 불용어 처리하며, 1이하의 실수를 입력하면 해당 비율을 초과해 등장한 단어를 불용어 처리한다. 반대로 min_df는 전체 문서 중 일정 횟수 미만으로 등장한 단어를 불용어 처리한다는 뜻이다.

vocabulary는 미리 구축한 단어사전이 있자면, 해당 단어 사전을 사용한다. 만약 입력하지 않는다면 TF-IDF 학습 시 자동으로 구축된다.

smooth_idf(idf 분모처리)는 IDF 계산 시 분모에 1을 더한다.

예를 들어

```
# 6.2 TF-IDF 계산

corpus = [
    "That movie is famous movie"
    "I like that actor",
    "I don't like that actor"
]
tfidf_vectorizer = TfidfVectorizer()
tfidf_vectorizer.fit(corpus)
tfidf_matrix = tfidf_vectorizer.transform(corpus)
```

```
print(tfidf_matrix.toarray())
print(tfidf_vectorizer.vocabulary_)


출력
[[0.26820807 0.         0.37695709 0.37695709 0.26820807
0.37695709
  0.37695709 0.53641614]
 [0.44832087 0.63009934 0.         0.         0.44832087
0.
  0.         0.44832087]]
{'that': 7, 'movie': 5, 'is': 3, 'famous': 2, 'moviei':
6, 'like': 4, 'actor': 0, 'don': 1}
```

를 하였을 때, 점수가 가장 높은 값을 세 개만 추려 색인으로 정리하면 [[2, 3, 5], [0, 4, 6], [0, 1, 4]]가 된다. 이를 단어 사전과 매핑하면 [[famous, is, movie], [actor, like, that], [actor, don, like]]가 된다. 이를 통해 문서마다 중요한 단어만 추출할 수 있으며, 벡터값을 활용해 문서 내 핵심 단어를 추출할 수 있다.하지만 출력 결과에서 확인할 수 있듯이 빈도 기반 벡터화는 문장의 순서나 문맥을 고려하지 않는다. 그
러므로 문장 생성과 같이 순서가 중요한 작업에는 부적합하다. 또한, 벡터가 해당 문서 내의 중요도를 의
미할 뿐, 벡터가 단어의 의미를 담고 있지는 않다.

▼ 10장

VIT(Vision Transformer)는 이미지를 자연어 처리 방식럼 분류해 보려는 시도에 의해 만들어짐. 즉 이미지 분류의 task이긴 하지만 CNN 모델의 합성곱 계층 방법을 사용하지 않고 트랜스포머 모델에서 사용되는 셀프 어텐션을 적용해 전체 이미지를 한 번에 처리한다. 하지만 ViT 모델은 이미지 패치가 순차적으로 입력되어(sequential하게) 구조의 이미지 특성을 온전히 반영한다고 보기 어렵기 때문에 이를 해결하기 위해 물체의 크기나 해상도를 계층적으로 학습하는 스윈 트랜스포머(Swin Transformer)와 CvT(COnvolutional Vision Transfoermr) 모델이 제안됐다.

- 스윈 트랜스포머는 local window를 활용해 각 계층의 어텐션이 되는 패치의 크기와 개수를 다양하게 이미지의 특징을 학습한다.viT 구조와 비교할 때 기존 셀프 어텐션을 local window 안에 어텐션, 로컬 윈도 간의 어텐션으로 수행하여 이미지의 특징을 계층적으로 학습시킨다. 이 어텐션 함수에는 상대적 위치 편향을 반영하여 어텐션 값 자체에 위치적 정보를 포함시킨다.